

Estimating Trajectory from an Inertial Measurement Unit

Charles Ciampa

Abstract—This lab compares estimating a trajectory using only an IMU to the trajectory derived from GPS. Using ROS2 Jazzy for data acquisition and custom message protocols, two datasets were gathered: one for calibrating the magnetometer and another consisting of an approximately 18-minute and 40-second drive. The heading for the drive was determined using a weighted average complementary filter with an α of 0.98. The velocity was estimated by setting the velocity to zero when the GPS read as stationary, estimating the sensor to be 10 cm away from the center of mass of the car, and using a high-pass filter on the x-acceleration data with a cutoff frequency of 0.1 Hz. By combining these estimates, a trajectory was generated. The IMU-estimated path maintained a position within 2 m of the GPS trajectory for 1.425 s from the start and 6.825 s from the end. Additionally, it was able to visibly maintain the shape of the GPS trajectory, albeit with some distortion. These results indicate that, with the setup used in this lab, an IMU would suffice to estimate the trajectory in the intervals between GPS measurements.

I. INTRODUCTION

DETERMINING one's position on Earth has been a challenge for millennia and continues to be an ongoing pursuit. Knowing one's position enables many applications, such as navigating across vast distances, helping people find their way in unfamiliar places, and performing precise measurements for construction sites. Historically, the positions of stars in the sky were used to determine location. Since then, vast advancements in physics and technology have been achieved, allowing for space travel, incredibly accurate time measurements, and the transmission of information over radio signals—all of which are instrumental for our current best and most widely used method for determining location: the Global Positioning System (GPS).

In addition, we have found ways to gather movement information using technology shrunk down to incredibly small sizes. This technology is called an Inertial Measurement Unit (IMU). Manufacturers have found ways to mass-produce IMUs in such quantities that the price has been driven so low that they are now integrated into a large number of electronics, including the majority of mobile phones.

The integration of both GPS and IMUs into phones has allowed for navigation capabilities never seen before. Now, we are able to estimate one's location and trajectory through space with meter-level precision using just a phone. In this lab, the goal is to estimate a car's trajectory using data gathered from an IMU and to compare it to data gathered from GPS.

II. METHODOLOGY

A. Data Collection

ROS2 Jazzy was used to run two nodes which read from two serial ports and published the GPS (Listing 1) and IMU (Listing 2) information using custom messages to the topics `/gps` and `/imu`, respectively. One node is responsible for reading GPS information from its serial port, and the other node reads IMU information from the serial port for the VN-100. The devices publish data to the serial port as a string structured under the NMEA 0183 standard specification. Each port publishes different forms of information containing various standards unique to the device. To extract the information from these strings, the data is separated by commas; the first part of the string indicates which standard is being published, and the end of the string contains information to validate the data. However, for this lab, it is not necessary to validate the data checksums as the transmission distance is short enough that invalid data is very unlikely.

For the GPS information, a GPGLA string is used. The GPS information was recorded from a GPS puck provided for the lab. [1] contains the exact details on how the GPGLA string is structured; refer to those references if interested in the specific breakdown of the string. Once the information is extracted, the latitude and longitude are converted into UTM coordinates using the UTM Python package, which provides the location in meters inside the respective zone. [2] is the repository containing the code for the UTM package and provides further information on its operation. This conversion allows all calculations to be performed assuming a Cartesian coordinate system, which simplifies the remainder of the analysis.

```
1 std_msgs/Header header
2 float64 latitude
3 float64 longitude
4 float64 altitude
5 float64 utm_easting
6 float64 utm_northing
7 uint8 zone
8 string letter
9 float64 hdop
10 uint8 fix_quality
11 string gpnga_read
```

Listing 1. Custom GPS Message

For the IMU information, a VNYMR string is used. The IMU data was recorded from a VN-100. [3] contains more information on the VN-100 and the VNYMR string. Since the IMU does not provide a timestamp with the data, the computer's clock is used to timestamp the data instead. For the

linear acceleration and angular velocity, the data was simply extracted and saved to the message. The magnetic data was slightly more complicated as it was provided in Gauss, which needed to be converted to Tesla; this conversion is performed by dividing the Gauss value by 10,000. The orientation is also complex because the VNYMR string provides rotation as Euler angles; however, ROS2 only supports angles in quaternions, as quaternions do not suffer from gimbal lock—an issue to be avoided in robotics. While the mathematics to convert from Euler angles to quaternions is well understood, it is complex, so SciPy’s Rotation class [4] was used for the conversions.

```
1 std_msgs/Header header
2 sensor_msgs/Imu imu
3 sensor_msgs/MagneticField mag_field
4 string raw
```

Listing 2. Custom IMU Message

The nodes were started using a launch file which activates both the GPS and IMU nodes. [5] is the repository containing the code ran to extract and publish this information. Refer to the `gps_driver`, `vn_driver`, and `custom_msg` directories inside the `/src` directory for the source code.

The data was recorded using the command `ros2 bag record /gps /imu`, which records the `/gps` and `/imu` topics. Two separate bag files were recorded for this lab. For both, the data was recorded in a car with the GPS puck on the roof and the IMU inside the vehicle.

The first bag was recorded by driving around the circle on Forsyth Street at Ruggles Station (Ruggles Circle) five times. The GPS data is ignored for this bag, as this dataset is used solely to calibrate the magnetometer.

The second bag is the primary dataset of interest. The recording started and ended at Ruggles Circle. The lab requirements specified a drive of around 2–3 km with a minimum of 10 turns for at least 10 minutes. The calibration for the magnetometer derived from the first bag is used to calibrate the magnetometer readings and determine the headings for this data.

For this lab, all GPS recordings were done inside UTM zone 19T; therefore, all Northing and Easting values are from within this zone. Approximately 18 minutes and 40 seconds of data were gathered for the drive of interest. Refer to the lab5 folder in [5] for the data and analysis as well as generated graphs.

III. ANALYSIS

A. Estimating the Magnetic Heading

To calibrate the magnetometer, data from the calibration drive was used. A critical requirement for calibrating the magnetometer is obtaining magnetic data from all orientations. The more data obtained for each direction, the better the resulting calibration. To perform the calibration, a least-squares approach was used, implemented via SciPy’s `least_squares` function. Equation 1 shows how the calibrated magnetic field is calculated in 3 dimensions. Equation 2 is a simplified version of Equation 1 used for this lab. In this 2D version, the Z-axis is ignored since the IMU should remain mostly

flat throughout the data gathering. The car should not pitch more than a small amount, given that the steepest road in Massachusetts does not achieve a gradient higher than 21.7% [6]; if the pitch were to exceed this, something has gone horribly wrong. Additionally, learning fewer variables allows the system to arrive at a solution faster.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} C_{00} & C_{01} & C_{02} \\ C_{10} & C_{11} & C_{12} \\ C_{20} & C_{21} & C_{22} \end{bmatrix} \cdot \begin{bmatrix} M_X - B_0 \\ M_Y - B_1 \\ M_Z - B_2 \end{bmatrix} \quad (1)$$

$$\begin{bmatrix} X \\ Y \end{bmatrix} = C \cdot (M - B) = \begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} \cdot \begin{bmatrix} M_X - B_0 \\ M_Y - B_1 \end{bmatrix} \quad (2)$$

In both Equation 1 and 2, the B variable is the hard iron offset and the C variable is the soft iron correction. The hard iron offset is due to an external magnetic force separate from Earth’s magnetic field which moves with the magnetometer, giving it a permanent offset from the center. The soft iron correction is a matrix that corrects distortions due to ferromagnets, which can distort Earth’s magnetic field and other external magnets. To obtain a solution, the fit was set to that of a circle. The goal is to minimize Equation 3, as this implies all points lie on a circle. However, attempting to minimize the entire equation at once often fails to find a best fit.

$$\min \left\| \begin{bmatrix} X \\ Y \end{bmatrix} \right\| - 1 \quad (3)$$

The calibration parameters, consisting of the hard iron offset vector $\mathbf{B}$ and the soft iron matrix $\mathbf{C}$, are determined via a two-stage least squares optimization process.

First, the hard iron offset $\mathbf{B}$ is solved by minimizing the Euclidean distance of the magnetic field measurements from the center. The objective function for this stage is given by:

$$\min_{\mathbf{B}} \sum_i \sqrt{(M_{X,i} - B_0)^2 + (M_{Y,i} - B_1)^2} \quad (4)$$

where $\mathbf{M} = [M_X, M_Y]^T$ represents the raw magnetometer data. Solving this centers the data around the origin.

Next, the soft iron matrix $\mathbf{C}$ is computed while holding $\mathbf{B}$ constant. The goal is to map the centered data to a unit circle (or sphere) by minimizing the deviation of the transformed magnitude from unity:

$$\min_{\mathbf{C}} \sum_i (\|\mathbf{C}(\mathbf{M}_i - \mathbf{B})\| - 1)^2 \quad (5)$$

A standard least squares solution for $\mathbf{C}$ may yield a valid mathematical fit that inadvertently reflects the input data (a “flipped” solution). To prevent this, the algorithm verifies the determinant of the resulting matrix, requiring $|\mathbf{C}| > 0$. Since the optimization is initialized with a random guess, the solver is iteratively re-run until a solution with a positive determinant is found.

Due to the rotational ambiguity inherent in solving for $\mathbf{C}$ (where infinite solutions exist on the unit circle), an additional

regularization term is required to maintain the general orientation of the sensor. This term is applied only to the first data point and is added to the minimization cost:

$$\text{Penalty} = \frac{\left(\arctan\left(\frac{M_{Y,0}-B_1}{M_{X,0}-B_0}\right) - \arctan\left(\frac{Y_0}{X_0}\right) \right)^2}{10,000} \quad (6)$$

The divisor of 10,000 ensures this factor acts as a weak constraint, sufficient to preserve directionality without compromising the primary shape-fitting optimization.

Figure 1 shows the calibrated magnetic field with the calculated C and B values. As can be seen, the calibrated magnetic field closely matches an ideal circle and has maintained the direction of the magnetic field. Since a car is made almost entirely of metal and the IMU is inside this essentially metal cage¹, a significant amount of distortion in the magnetic data is expected. This is visibly seen as a large hard iron offset, which can result from two sources: the frame of the car likely contains iron and could have obtained its own magnetic field, and/or proximity to speakers, which contain magnets to produce audio. The soft iron distortions are definitely a result of the iron car frame, though they were not as severe as expected; this is likely due to the frame covering the entire sensor somewhat evenly, potentially reducing the net effect of the soft iron distortion.

From the calibrated magnetic data, heading can be found by simply taking the inverse tangent of the calibrated x and y magnetic values. For this lab, NumPy's `arctan2` function [7] is used since it can handle issues that would arise from simply doing the inverse tangent. With the arc-tangent, the heading can be found for any point in the data. Using the parameters found from the calibration dataset on the actual drive dataset, the magnetic heading can be calculated.

Figure 2 shows both the heading from the uncalibrated magnetic data and the calibrated magnetic heading. As can be seen, the calibrated magnetic heading is quite close, though the calibrated heading covers the entire range of $[-\pi, \pi]$, which shows that it is a good fit.

B. Gyroscope Yaw Estimate

Figure 3 shows the estimated heading from the Z-Axis angular velocity readings. As can be seen, this was actually incredibly stable; however, the biggest issue with the gyro readings is that they tend to drift over time. Therefore, while it seems better than the magnetic yaw estimate (Fig. 2), the magnetic yaw will not drift and will be closer to the true value over the long term. This is why filters are used to combine the magnetic and gyro data to get better estimates.

C. Combining Magnetometer and Gyroscope Yaw Estimates

The filter used to combine the magnetic yaw and gyro yaw estimates is called a complementary filter. The goal of the complementary filter is to take advantage of the magnetic

heading (which has noise but no drift) and the stability of the gyroscopic measurements (which are smooth but drift over time). To do this, two separate approaches were used in this lab, as shown in Figure 4.

The first approach in Figure 4 is a combination of a low-pass filter for the magnetometer data and a high-pass filter for the gyroscopic data. A low-pass filter removes frequencies greater than a specified frequency, while a high-pass filter removes frequencies lower than a specified frequency. These filters are often used in audio for speakers to split up frequencies of sound for the specific driver that can handle the frequency best². While this is done electronically in audio, it can be simulated in code, which is what is done for this lab. SciPy's functions `butter` and `sosfilt` are used to achieve this. Refer to [8] and [9] for more information on these functions. In this case, a cutoff frequency of 0.1 Hz is used for both the low-pass and high-pass filters. As can be seen in the top two graphs, this does a good job of removing the noise from the magnetic data and the drift from the gyroscopic data. To combine these filters, it is a simple addition of their filtered outputs, which gives the blue line in the bottom left graph of Figure 4.

The second approach achieves the same goal as the previous complementary filter but in a different manner. It uses an iterative approach where the predicted new angle from the gyroscope is combined with the magnetic estimate for the angle. This is shown in Equation 7, where θ_m is the magnetic heading, ω is the angular velocity around the Z-Axis, and $\alpha \in [0, 1]$ is a hyperparameter.

$$\theta_{t+1} = \alpha(\theta_t + \omega_t \Delta t) + (1 - \alpha)\theta_{m,t} \quad (7)$$

This has the natural effect of prioritizing the gyroscope for more immediate changes; however, after many steps, the magnetic heading estimate will correct any drift. This effect can be adjusted with the hyperparameter α : the higher the α , the longer the magnetic heading takes to affect the input of the gyro. For this lab, an α of 0.98 was used. While this seems large, in one second of data there are 40 points; therefore $1 - 0.98^{40} \approx 55\%$. This means after a second, the majority of the angle estimate is being driven by the magnetic heading. While the formula is mostly correct, an adjustment has to be made to the magnetic heading since $\theta_m \in [-\pi, \pi]$. If θ_m increases near π , it will flip to $-\pi$, appearing to decrease by -2π when it has actually increased by a small amount. This means if the estimate for the new angle from the gyroscope is still close to π but the magnetic heading is close to $-\pi$, averaging them will result in a value close to 0, when it should be closer to π . Therefore, a check is done at each step to see if the difference between the new yaw estimate from the gyroscope and the magnetic heading is larger than π . If so, an offset of 2π is added or subtracted from the magnetic heading so the difference is minimized and the average between the two is calculated correctly.

¹This is also why being inside your car during a thunderstorm is the safest place, since the metal of the car will redirect any lightning shocks to the ground, protecting anything inside.

²Woofers are best for lower frequencies and tweeters are best for higher frequencies. In most speaker arrays you will find a combination of these two as most speakers are not capable of covering the entire range of frequencies best.

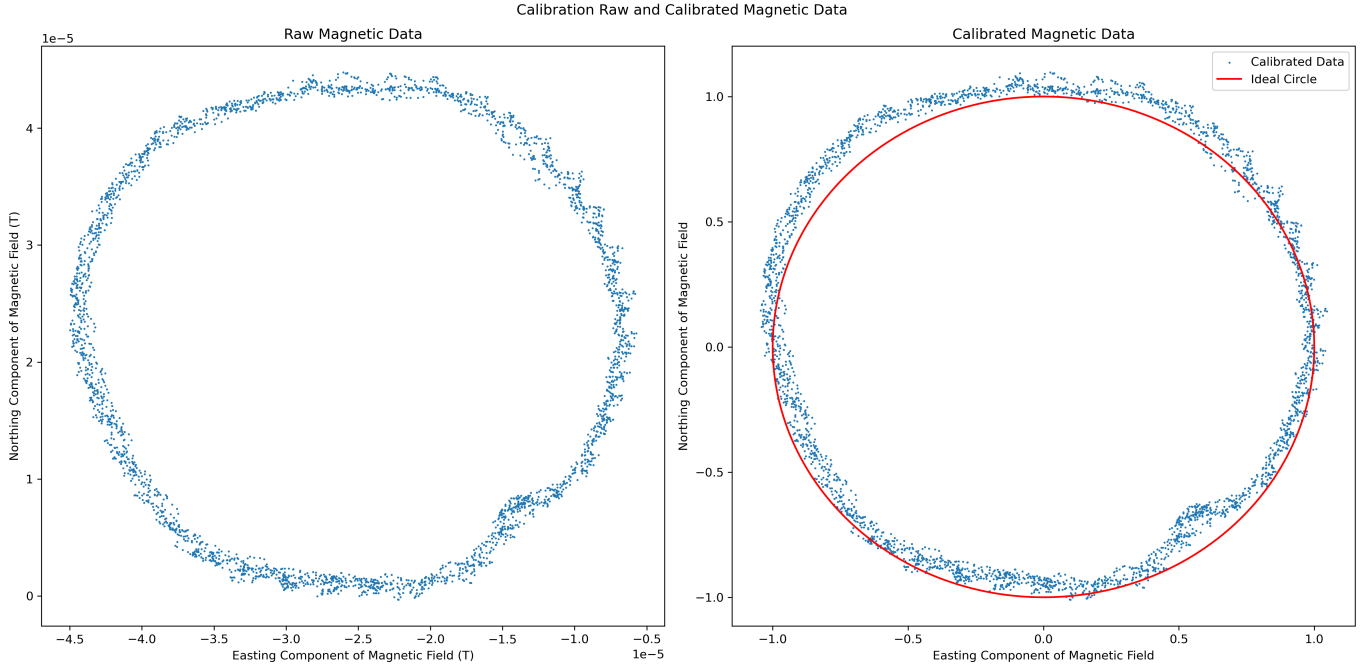


Fig. 1. The left side shows the raw magnetic data from the IMU. The right side shows the calibrated magnetic data along with an ideal circle.

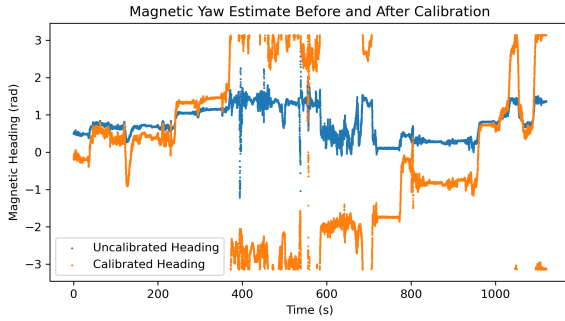


Fig. 2. The magnetic heading over the approximately 19 minute drive.

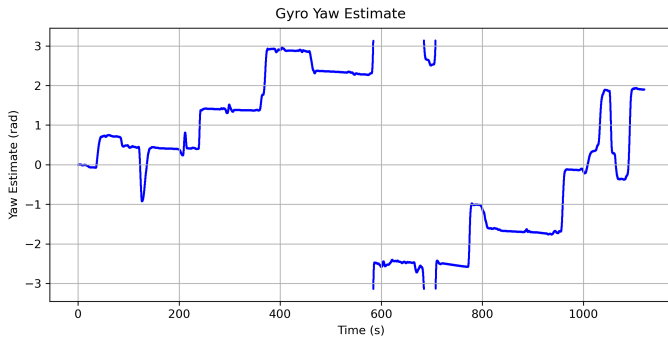


Fig. 3. The estimated yaw readings over time were calculated by summing up all the $\omega \times \Delta t$ at each timestep, where Δt is the rate of the sensor readings and ω is the angular velocity around the Z-Axis.

The issue of "jumping around" is specifically observed in the first complementary filter near 600s, where there appears to be a full revolution; however, the gyroscope does not corroborate this in Figure 3, meaning it is likely due to noise in the magnetometer. As can be seen, this is corrected in the second complementary filter (the purple line in the bottom left graph of Figure 4). The second complementary filter also more closely matches the IMU's estimated heading, indicating it is likely a better estimate. Therefore, for the yaw estimate, the second complementary filter or the IMU's yaw estimate are best trusted for navigation. Later in the lab, when estimating the path of the car from the IMU, the second complementary filter yaw estimate will be used.

D. Estimating Forward Velocity

To estimate the forward velocity of the car from the IMU, since the X-Axis should have been facing the direction of movement throughout the data collection, integrating the acceleration on the X-Axis should give a good estimate of the car's velocity. While in theory this should be a decent estimate, several issues arise. First, there are two main sources of velocity drift: if the sensor is tilted at any angle, a constant acceleration will be added due to gravity; and when stationary, vibrations from the engine slightly shake the IMU, causing small accelerations to accumulate. Lastly, since the IMU is not at the center of the car, there is an additional acceleration observed when turning.

To address the issue of velocity drift, two main approaches are used. First, to solve the tilt issue, while using the pitch of the sensor to remove the gravity component should theoretically work, in practice, it is ineffective. Therefore, the approach used for this lab was using a high-pass filter on the

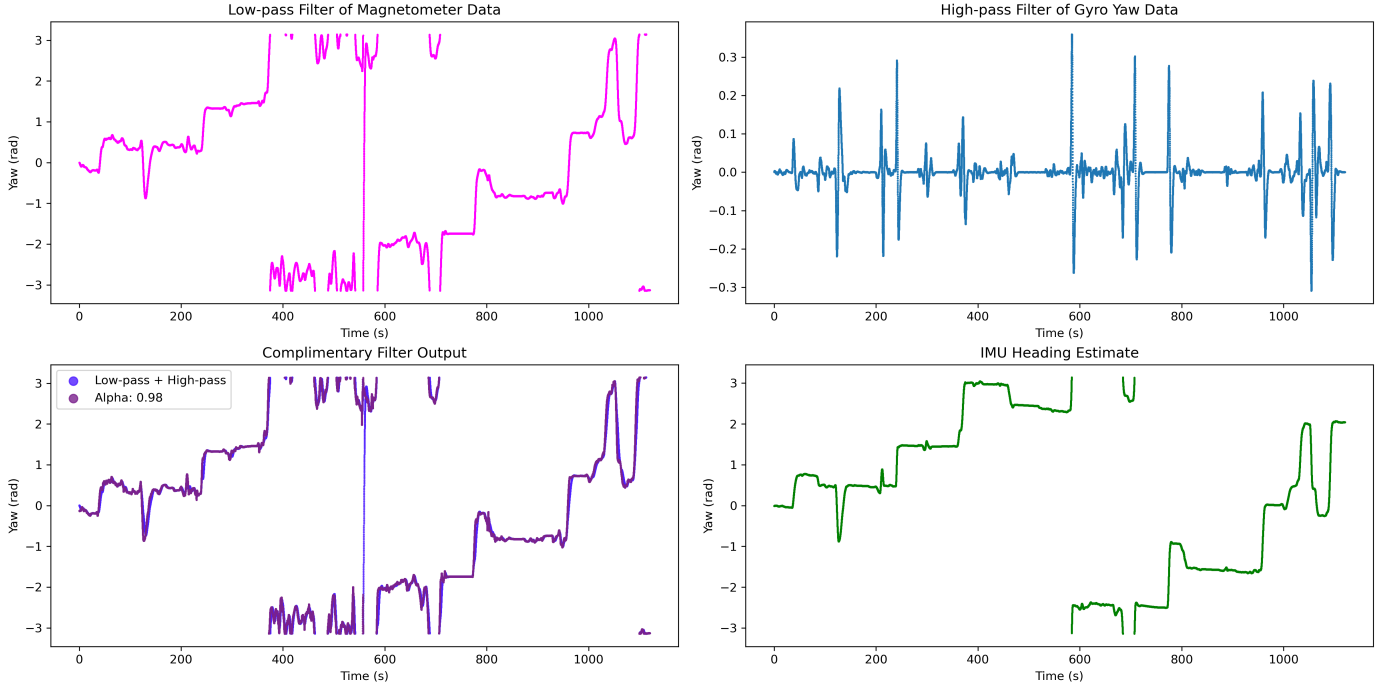


Fig. 4. The graphs show the low-pass and high-pass filters for the magnetometer data and gyroscopic data respectively. Additionally, the bottom left graph shows two different magnetometer and gyroscopic complementary filters yaw estimates where the blue line shows the low-pass and high-pass combined complementary filter and the purple line shows the iterative complementary filter. The bottom right graph shows the estimate for the yaw outputted from the IMU.

acceleration data, where the cutoff frequency was set to a low frequency of 0.1 Hz. This value was chosen assuming that the slowest a car might accelerate would be from 0 ms^{-1} to 30 ms^{-1} in 30 s, which would be an acceleration of 1 ms^{-2} . Therefore, any value below 1 Hz should suffice. A cutoff of 0.1 Hz provides sufficient room for slower accelerations and was also very effective at removing the offset due to gravity.

As for the velocity drift when the car is stationary, this was solved by detecting when the GPS velocity was measured to be zero and then forcing the estimated velocity to zero. While many different approaches could be used to detect when the car is stationary (such as attempting detection via convolutions or adjusting for the pitch of the sensor), these still experienced drift; to make them work well, much more advanced techniques would be required.

For the issue of the additional acceleration observed from the sensor being off-center, a correction can be calculated. The formulas below show what the observed accelerations on both the X-Axis and Y-Axis are during turns. Equations 8 and 9 show the observed acceleration given the actual acceleration of the car, a_e , the turning radius of the car, r , the angular velocity of the car, ω , the distance of the IMU from the center of the car, x_c , and the y velocity of the IMU from the center of the car, v_y .

$$a_{x,obs} = a_e - \frac{d|\omega|}{dt}r + \omega^2 x_c \quad (8)$$

$$a_{y,obs} = \frac{d|\omega|}{dt}x_c + 2|\omega|v_y + \omega^2 r \quad (9)$$

First, it is necessary to test if adjustments for the center are needed at all. If there is no offset, then $v_y = 0$ and $x_c = 0$. If a constant angular velocity is assumed, the equations simplify significantly, with $a_{x,obs} = a_e$ (which is expected) and $a_{y,obs} = \omega^2 r = \omega v_x$. A simple integration of the x-acceleration with a high-pass filter³ was performed to get velocity, then multiplied by the measured angular velocity to get an estimate of the expected y-acceleration. The magnitude of this calculated y-acceleration and the observed y-acceleration (also with a high-pass filter cutoff of 0.01 Hz) were found to have a correlation of ~ 0.39 with a Root Mean Square Error (RMSE) of $\sim 0.80 \text{ ms}^{-2}$. While these values indicate that this is a decent approximation, it is clear that improvements can be made and that the correction should be added.

The goal is to solve for a_e , which is the actual acceleration of the car. Equation 10 shows how rearranging Equation 8 allows a_e to be found. However, the only unknown here is the radius, r .

$$\begin{aligned} a_{x,obs} &= a_e - \frac{d|\omega|}{dt}r + \omega^2 x_c \\ a_e &= a_{x,obs} + \frac{d|\omega|}{dt}r - \omega^2 x_c \end{aligned} \quad (10)$$

Luckily, by rearranging Equation 9, the radius can be solved for as seen in Equation 11. However, before getting r , v_y needs to be solved for in Equation 9. Since the distance from the center of the car is known, it is possible to estimate v_y with $v_y = x_c \omega$, as this should give the y velocity relative to the center of the car.

³A cutoff frequency of 0.01 Hz

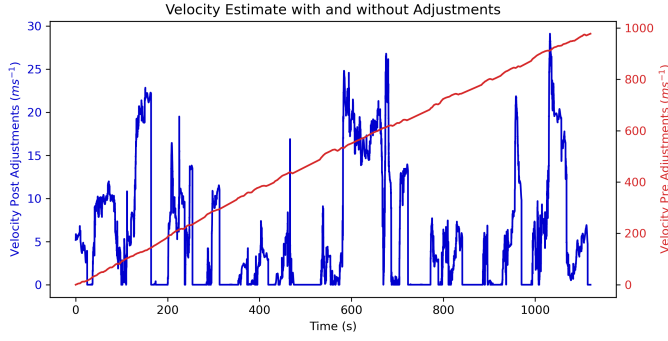


Fig. 5. The red line is the velocity estimate before any adjustments were made. The blue line shows the velocity estimate after the adjustments were made.

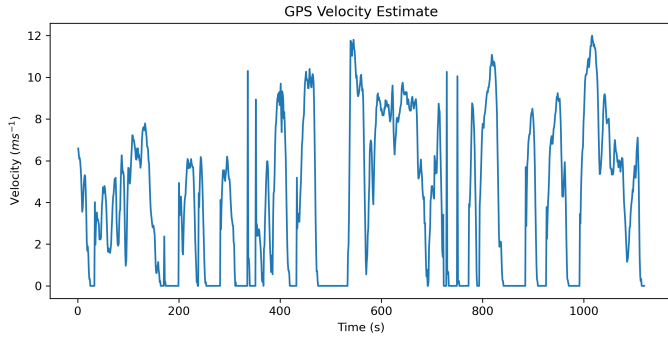


Fig. 6. Velocity Estimates From GPS by dividing the magnitude of change in position divided by the time between the measurements.

$$\begin{aligned}
 a_{y,obs} &= \frac{d|\omega|}{dt} x_c + 2|\omega|(x_c \sin \omega) + \omega^2 r \\
 \omega^2 r &= a_{y,obs} - \frac{d|\omega|}{dt} x_c - 2|\omega|(x_c \sin \omega) \\
 r &= \frac{1}{\omega^2} (a_{y,obs} - \frac{d|\omega|}{dt} x_c - 2|\omega|(x_c \sin \omega))
 \end{aligned} \quad (11)$$

Using this calculated radius value, the actual acceleration of the car can be estimated. It can be seen that when comparing the velocity before and after adjustments, the velocity after adjustments is clearly more accurate. For the sake of this lab, an x_c of 0.1 m was assumed since there is no way to know the exact displacement without an actual measurement.

Figure 5 corroborates the drift in the velocity, having a constant acceleration added to it which makes it appear to be a straight line. As can be seen, the adjusted velocity much more closely matches that from the GPS. To fix this, many different methods can be employed to align the velocity more closely with the GPS velocity estimates seen in Figure 6.

E. Estimating Trajectory

To estimate the trajectory, using the heading estimate and the velocity estimate, we integrate the x and y components of displacement over time by taking the cumulative sum of the velocity projected onto each axis (using cosine for x and sine for y) multiplied by the time step. This gives the trajectory for the IMU shown in the right graph of Figure 7.

As can be seen in Figure 7, while the IMU estimated trajectory is clearly not an exact match to the GPS trajectory, it was able to reconstruct the path. The main differences between the two are the distances; additionally, for some reason, after the first turn, the system believes there to be a large jump in its velocity. However, the direction seems to match very closely. The IMU is able to maintain within 2 m of the GPS trajectory for a total of 1.425 s. This is not particularly long, but interestingly, if calculated from the end of the drive, the IMU maintains within 2 m for a much longer time of 6.825 s. While still not perfect, this shows that drift was a large issue and that there are likely many portions which closely match the GPS trajectory. Given this performance, using this algorithm, one can expect to go a few seconds without another fixed position and still maintain a good estimate of trajectory, which for this setup would be sufficient time to acquire a new GPS location.

IV. CONCLUSION

From driving in a circle, the magnetometer was calibrated to closely match a circle with very little variation, showing that it can give a good heading estimate. This was corroborated when applied to the real driving magnetic data as seen in Figure 2 and when compared to the IMU's yaw estimate from the gyroscope. The low-pass + high-pass complementary filter was effective and gave a result that closely matches that given from the IMU; however, the weighted average complementary filter performed much better.

Estimating the velocity was overall the most difficult part of the project as drift was a significant issue. Additionally, noise from the engine and the sensor not being perfectly flat throughout the data collection caused the velocity estimates to be suboptimal, even after using the GPS to detect when stationary. Even with these challenges, the velocity estimates were not terrible and decently matched the GPS velocity estimates when comparing graphs in Figures 5 and 6. Without adjusting the velocity for the offset from the center of the car, an RMSE of $\sim 0.80 \text{ ms}^{-2}$ was observed.

The IMU's trajectory estimate is good given the simplicity of the techniques used to estimate it and the imperfect conditions of the data gathered. The IMU was able to maintain within 2 m of the GPS trajectory for a total of 1.425 s from the start and a total of 6.825 s from the end of their trajectories. Given these two metrics, it is likely that a rate of 1 Hz for GPS data would be sufficient to maintain a good trajectory in conjunction with the IMU.

Many improvements could be made to this system. One easy improvement would be to gather some stationary data in the car and to start the drive from a stationary point. This would allow for a better approximation of gravity to remove it from the signal. An additional improvement would be a better setup for the IMU—one which would keep it rigidly fixed relative to the car, meaning any deviations would be due to the car itself. It would also be interesting to test with the IMU on top of the car to see if this would improve the magnetic readings. Additionally, using a Kalman filter would improve estimates over time as it is able to minimize the uncertainty

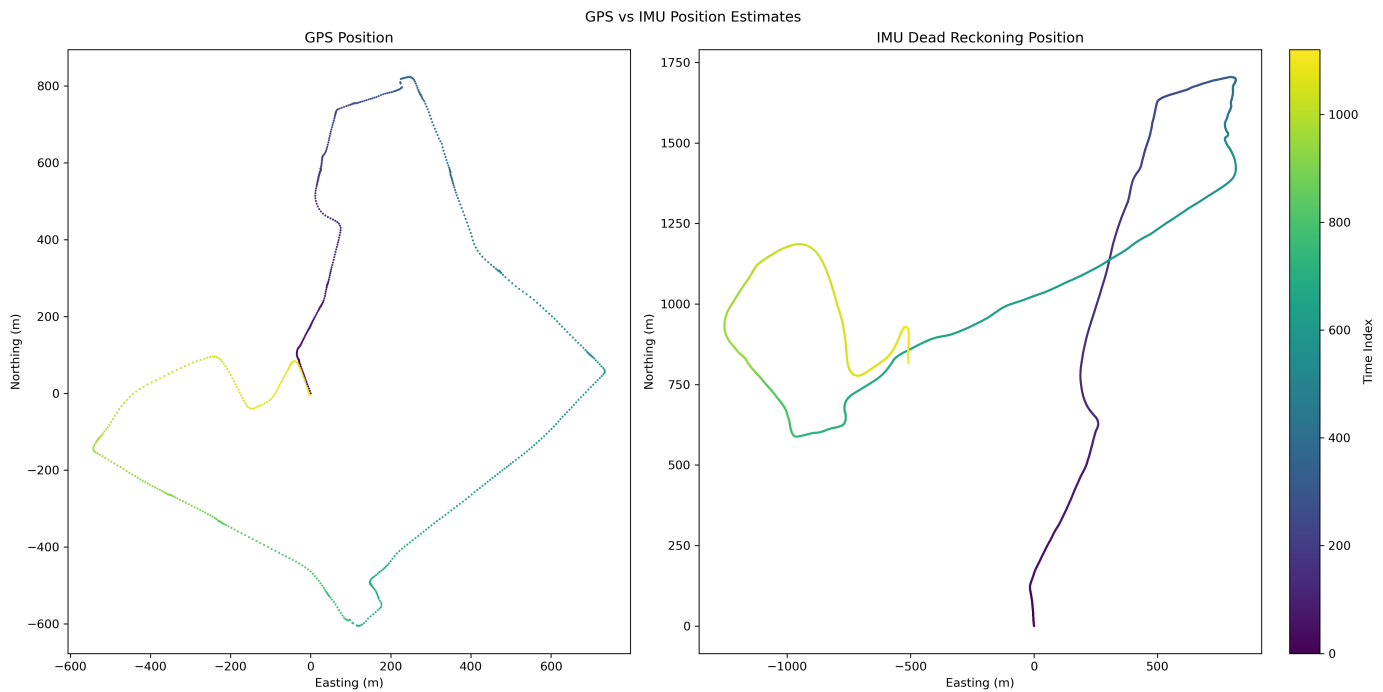


Fig. 7. Both graphs show the estimated trajectory from the start of the drive being at the origin. The color shows at what time in the trajectory the trajectory estimate is at.

of the measurements. Another interesting experiment would be to see if performing this with multiple IMUs would improve the estimate of the velocity. It would also be worthwhile to test with an electric car; since these do not have engines, they will be stationary at a stop and there should be significantly less noise in the sensor readings, likely yielding much better velocity estimates. Though it would be important to watch the magnetometer readings as they may be affected by the electric car's motors, which create magnetic fields to run.

ACKNOWLEDGMENT

Google Gemini 3 Pro was used for formatting help and as well as with clarity (<https://gemini.google.com/share/4725728680d8>). Claude models Sonnet 4.5 and Opus 4.5 were used at times to help with the coding of analysis with their uses noted in the code as comments. Lastly Google Gemini 3 Pro was used for final edits to fix grammar mistakes and improve the flow (<https://gemini.google.com/share/97187294669c>).

REFERENCES

- [1] "GPVGA." [Online]. Available: <https://docs.novatel.com/OEM7/Content/Logs/GPVGA.htm>
- [2] T. Bieniek, B. van Andel, and T. I. Bø, "Turbo87/utm," Sep. 2025, original-date: 2012-12-29T19:02:49Z. [Online]. Available: <https://github.com/Turbo87/utm>
- [3] "VectorNav's VN-100 IMU/AHRS, the world's most trusted surface mount solution." [Online]. Available: <https://www.vectornav.com/products/detail/vn-100>
- [4] "Rotation — SciPy v1.16.2 Manual." [Online]. Available: <https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.spatial.transform.Rotation.html>
- [5] C. Ciampa, "CharlesC03/EECE5554." [Online]. Available: <https://github.com/CharlesC03/EECE5554>

- [6] "BRENTACOL | Kingsley Hill Road." [Online]. Available: <https://www.brentacol.com/h/kingsley>
- [7] "numpy.arctan2 — NumPy v2.3 Manual." [Online]. Available: <https://numpy.org/doc/2.3/reference/generated/numpy.arctan2.html>
- [8] "butter — SciPy v1.16.2 Manual." [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html>
- [9] "sosfilt — SciPy v1.16.2 Manual." [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.sosfilt.html>